AI-BASED FIRE DETECTION SYSTEM USING CNN WITH MAIL ALERT NOTIFICATION

Internship Report Submitted to

AYYA NADAR JANAKI AMMAL COLLEGE, SIVAKASI.

affiliated to the **Madurai Kamaraj University, Madurai**, in partial fulfilment of the requirements for the award of the Degree Programme of

MASTER OF COMPUTER APPLICATIONS

Submitted by

ATHI RAMAR A

(Register No: 25PCA134)

Internship offered by

Company name with place

Mentored by

Mr.T. Jebastin M.C.A., M.PHIL.,

Associate / Assistant Professor

POST GRADUATE DEPARTMENT OF COMPUTER APPLICATIONS

AYYA NADAR JANAKI AMMAL COLLEGE

(Autonomous, affiliated to **Madurai Kamaraj University,** Re-accredited (4th Cycle) with 'A+' Grade (CGPA of 3.48 out of 4) by NAAC, Recognized as College of Excellence and Mentor Institution by UGC, Star College by DBT and Ranked 72nd at National Level in NIRF 2025 and DST-FIST (2023) Supported & An ISO 9001:2015 & ISO 21001:2018 Certified Institution)

SIVAKASI – 626 124.

July - 2026

Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.
Director,
Post Graduate Department of Computer Applications,
Ayya Nadar Janaki Ammal College (Autonomous),
Sivakasi.

CERTIFICATE

This is to certify that this "AI-BASED FIRE DETECTION SYSTEM USING CNN WITH MAIL ALERT NOTIFICATION" being submitted by ATHI RAMAR A (25PCA134), final year student of Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by him/her under the guidance of **Mr.T. Jebastin M.C.A., M.PHIL.,** with Qualification, Assistant/Associate Professor, Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi.

Place: Sivakasi, **Signature of the Director**

Date: **(Dr. R. VENGATESHKUMAR)**

GUIDE NAME with Qualification
Assistant/Associate Professor,
Post Graduate Department of Computer Applications,
Ayya Nadar Janaki Ammal College (Autonomous),
Sivakasi.

CERTIFICATE

This is to certify that this **"Internship on Project name"** being submitted by **Student name (25PCA1--)**, final year student of Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by him/her under my guidance and supervision. It is further certified that to the best of my knowledge, this Internship report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place : Sivakasi,
Date:

Signature of the Guide
(GUIDE NAME)

STUDENT NAME,

25PCA1--,

II MCA,

Post Graduate Department of Computer Applications,

Ayya Nadar Janaki Ammal College (Autonomous),

Sivakasi.

DECLARATION

This Internship report entitled **"PROJECT TITLE"**, has been carried out by me at the Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications.

I further declare that this Internship report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place: Sivakasi, **Signature of the Candidate**

Date: **(STUDENT NAME)**

ACKNOWLEDGEMENT

At the outset I thank **God** almighty for having brought, he will fulfilled power upon us in completing this Internship Programme.

I heartily express my sincere thanks to our College Management and our beloved Principal, **Dr. C. ASHOK, B.Sc., M.P.Ed., M.Phil., D.Y.MEd., Ph.D.,** for providing me this great opportunity and complete facility during this Internship Programme.

I heartily express my sincere thanks to **Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.** Director, Post Graduate Department of Computer Applications for his valuable advice and guidance throughout this Internship Programme.

I heartily express my sincere thanks to **GUIDE NAME with qualification, Associate/Assistant Professor**, Post Graduate Department of Computer Applications for his/her excellence guidance, encouragement and personal commitments manifested throughout this Internship Programme.

I extend my sincere thanks to the Team leader of **"Company Name with Place"** for their support throughout this Internship Programme.

I extend my sincere thanks to all our **department staff members** for their excellent support during this Internship Programme.

I also thank **my parents** for their encouragement and affection which helped us much to involve enthusiastically in our Internship Programme.

Place: Sivakasi, **STUDENT NAME**

Date:

Chapter I

Abstract

The AI-Based Fire Detection System using CNN with Mail Alert Notification is an intelligent solution for real-time fire monitoring that leverages deep learning to ensure rapid and accurate detection. The system captures live video streams from strategically placed cameras, with OpenCV handling frame extraction and preprocessing to enhance relevant visual features. A pre-trained Convolutional Neural Network (CNN) analyzes each frame to identify fire patterns, distinguishing actual fires from false positives caused by bright lights or reflections. Upon detection, the system immediately triggers audible alarms to alert nearby individuals, enabling prompt evacuation and emergency response.

In addition to on-site alerts, the system incorporates an SMTP-based email notification module that sends automated alerts to designated recipients. These notifications include the detection timestamp, a 10-second video clip of the fire, and predictive insights about its occurrence, ensuring timely awareness even when personnel are not present. By combining AI-driven fire detection with real-time alerts and video documentation, the system enhances situational awareness, reduces potential damages, and provides a proactive approach to fire safety in residential, commercial, and industrial environments.

Chapter 2

Introduction

2.1 About the Project

The AI-Based Fire Detection System using CNN with Mail Alert Notification is a smart solution designed to enhance fire safety by combining real-time surveillance with artificial intelligence. The system uses strategically placed cameras to continuously capture video footage, which is processed using OpenCV to extract important visual features. A Convolutional Neural Network (CNN) then analyzes these frames to accurately detect fire, distinguishing it from other bright or flickering objects. When a fire is identified, the system immediately triggers alarms to alert people in the vicinity, enabling quick evacuation and response.

In addition to local alerts, the system automatically sends email notifications to designated recipients through an SMTP module, including details such as the detection timestamp, a 10-second video clip, and predictive insights about the fire. All detected events are securely stored for future review and analysis. By integrating AI-based detection with real-time alerts and automated reporting, this project provides a proactive, efficient, and reliable solution for fire monitoring in residential, commercial, and industrial environments.

Chapter 3

Configuration Requirements

3.1 Hardware Requirements

Processor : Intel Dual Core Processor

RAM : 4 GB

Hard Disk : 500 GB

3.2 Software Requirements

- Front End : Python IDE 3.7.1, OpenCV
- Back End : TensorFlow

PACKAGES

TensorFlow

Image recognition consists of pixel and pattern matching to identify the image and its parts. Each image is a container of pixels which in turn are the combination of numbers. These numbers represent the color depth. Here the character recognition can be executed by this TensorFlow framework.

Scikit-learn

It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python.

Tkinter

Tk widgets can be used to construct buttons, menus, data fields, etc. in a Python application. Here the application is designed with button to upload the input image.

NumPy

NuAmPy can be used to perform a wide variety of mathematical operations on arrays. The comparision and prediction with the dataset can be implemented by using various mathematical calculation in recognizing the handwritten character input.

Matplotlib

Matplotlib is a cross-platform, data visualization and graphical plotting library for Python and its numerical extension NumPy.

Pillow

Pillow is a Python Imaging Library (PIL), which adds support for opening, manipulating, and saving images.

ipython

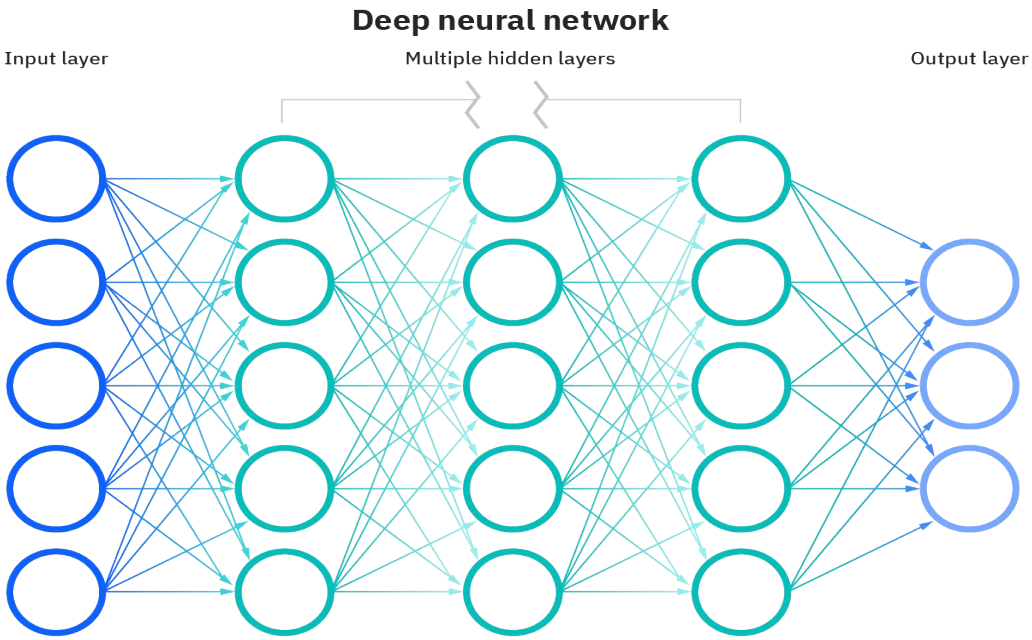
It is an interactive command-line terminal for Python. IPython offers an enhanced read-eval-print loop (REPL) environment particularly well adapted to scientific computing.

SpeechPy

SpeechPy is an open source Python package that contains speech preprocessing techniques, speech features, and important post-processing operations. It provides most frequent used speech features including MFCCs and filter bank energies alongside with the log-energy of filter-banks. The aim of the package is to provide projector's with a simple tool for speech feature extraction and processing purposes in applications such as Automatic Speech Recognition and Speaker Verification.

NEURAL NETWORKS

Neural network is a system inspired by human brain function, consists of neurons connected in parallel with the ability to learn. A basic design of neural network has input layer, hidden layer, and output layer. Neural networks, with their remarkable ability to derive meaning from complicated or imprecise data, can be used to extract pattern and detect trends that are too complex to be noticed by either humans or other computer techniques



CONVOLUTIONAL NEURAL NETWORKS

A special type Neural Networks that works in the same way of a regular neural network except that it has a convolution layer at the beginning. A convolutional neural network consists of an input and an output layer, as well as multiple hidden layers. The hidden layers of a CNN typically consist of convolutional layers, RELU layer i.e. activation function, pooling layers, fully connected layers and softmax layers.

About Algorithm

Convolutional Neural Networks

CNN are made up of a large number of interconnected neurons that have learnable weights and biases. In the architecture of CNN the neurons are organized as layers. It consist of an input layer, many hidden layers and an output layer. If the network has a large number of hidden layers the same are generally referred as deep neural networks. The neurons in the hidden layers of CNN are connected to a small

region (receptive field) of the input space generated from the previous layer instead of connecting to all, as in the fully connected network like Multi Layered Perceptron (MLP) networks. This approach reduces the number of connection weights (parameters) in CNN compared to MLP. As a consequence, CNN takes less time to train for the networks of similar size. The inputs to the typical CNN are two dimensions (2D) array of data such as images. Unlike the regular neural network the layers of a CNN are arranged in three dimensions (width, height and depth).

The followings are the type of layers which as commonly found in the CNNs.

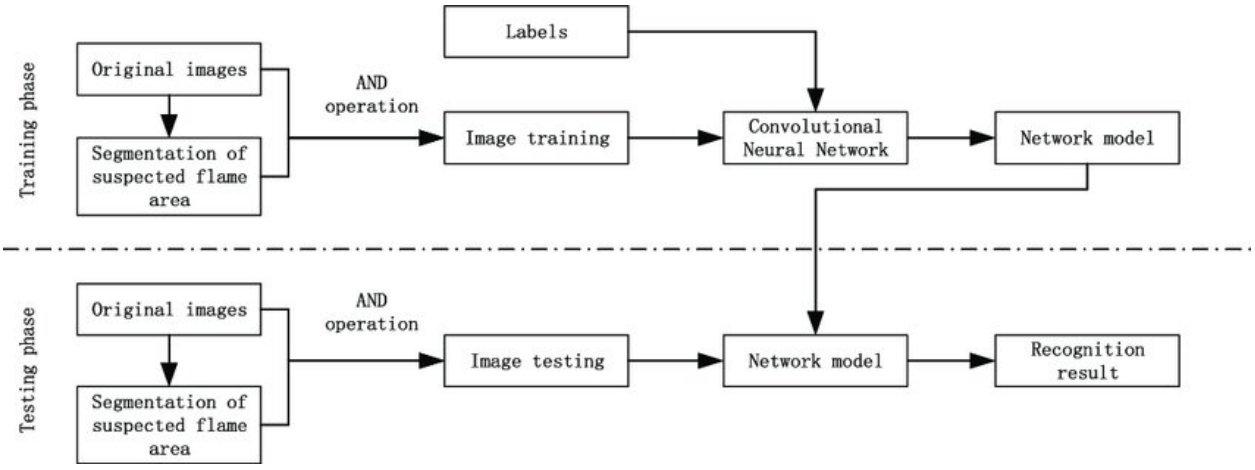
- Input Layer is basically a buffer to hold the input and pass it on to the next layer.
- Convolution Layer performs the core operation of feature extraction. It performs Convolution operation of the input data. The convolution operation is executed by sliding the kernel over the input, and performs sum of the product at every location. The step size with which the kernel slides are known as stride. Numerous convolution operations are performed on the input by using different kernel, which results in different feature maps, the number of feature map produced in a convolutional layer is also known as the depth of the layer.
- Rectified Linear Unit (ReLU) is an activation function used to introduce non linearity. It replaces negative value with zero, which can speed up the learning process. The output of every convolution layer is passed through the activation function.
- Pooling layer reduces the spatial size of each feature map, which in turn reduces the computation in the network. Pooling also uses a sliding window that moves in stride across the feature map and transform it into representative values. Min pooling, average pooling and max pooling are commonly used.

in the graph represents a mathematical operation, and each connection or edge between nodes is a multidimensional data array, or tensor. TensorFlow provides all

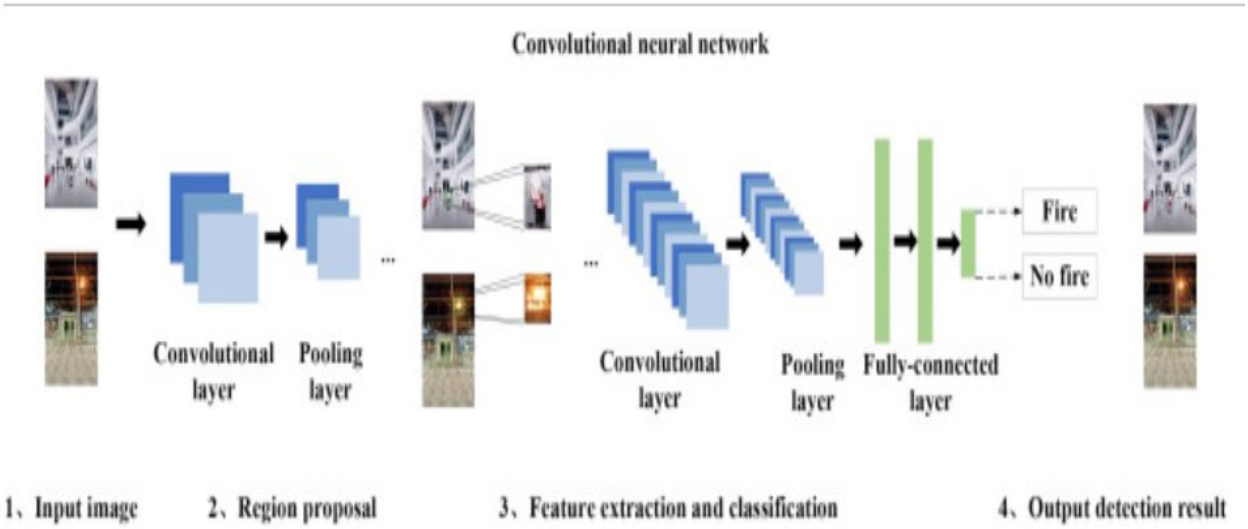
Chapter 4

System Design

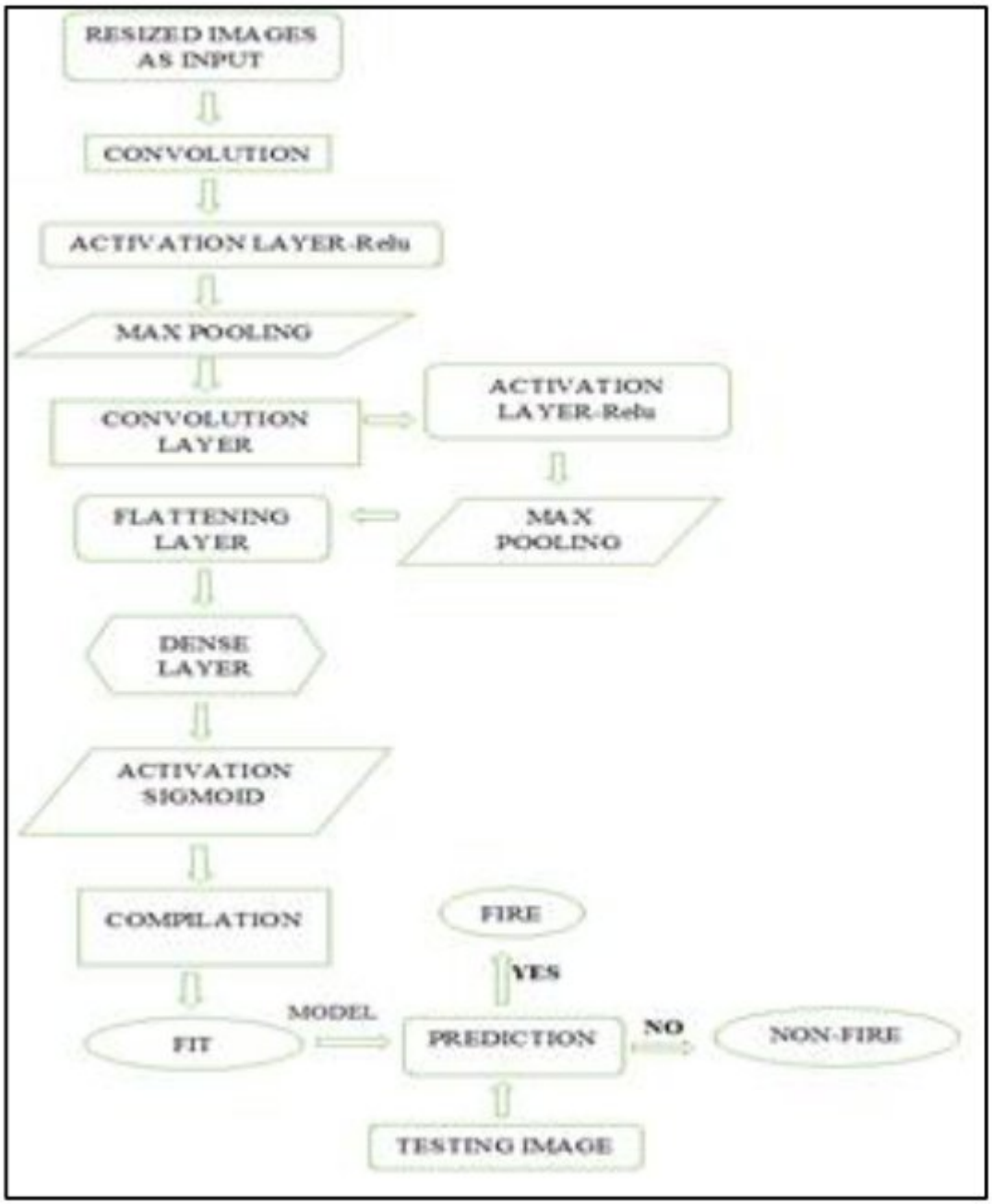
4.1 Data Flow Diagram



Working of CNN Algorithm



Flow Chart of CNN Training Model



About Algorithm

Convolutional Neural Networks

CNN are made up of a large number of interconnected neurons that have learnable weights and biases. In the architecture of CNN the neurons are organized as layers. It consist of an input layer, many hidden layers and an output layer. If the network has a large number of hidden layers the same are generally referred as deep neural networks. The neurons in the hidden layers of CNN are connected to a small region (receptive field) of the input space generated from the previous layer instead of connecting to all, as in the fully connected network like Multi Layered Perceptron (MLP) networks. This approach reduces the number of connection weights (parameters) in CNN compared to MLP. As a consequence, CNN takes less time to train for the networks of similar size. The input to the typical CNN are two dimensions (2D) array of data such as images. Unlike the regular neural network the layers of a CNN are arranged in three dimensions (width, height and depth).

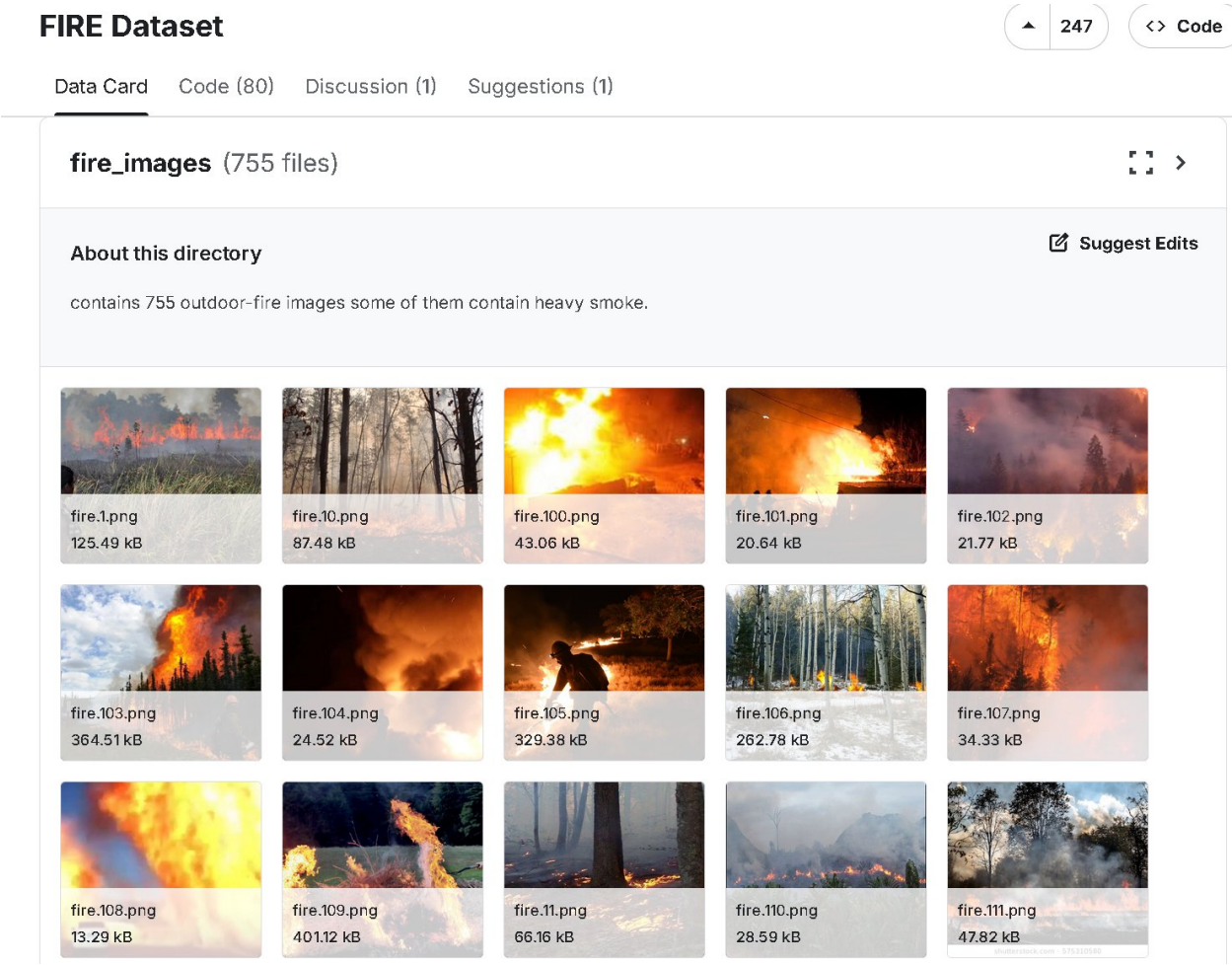
The followings are the type of layers which as commonly found in the CNNs.

- Input Layer is basically a buffer to hold the input and pass it on to the next layer.
- Convolution Layer performs the core operation of feature extraction. It performs Convolution operation of the input data. The convolution operation is executed by sliding the kernel over the input, and performs sum of the product at every location. The step size with which the kernel slides are known as stride. Numerous convolution operations are performed on the input by using different kernel, which results in different feature maps, the number of

Dataset

The fire dataset is taken from the below link

https://www.kaggle.com/datasets/phylake1337/fire-dataset



The dataset contains labeled images of fire and non-fire scenes, making it suitable for training deep learning models, particularly Convolutional Neural Networks (CNNs), to detect fire accurately. It provides a balanced dataset with diverse fire scenarios, including variations in lighting, background complexity, and fire intensity.

Dataset Structure

The dataset is organized into two main categories:

- Fire Images – Images containing fire in different environments.
- Non-Fire Images – Images without fire, used to train models to differentiate between fire and non-fire situations.

These images are stored in separate folders, making it easy to use for training, validation, and testing.

Data Composition

- Total Images: The dataset contains thousands of images, ensuring diversity for robust model training.
- Resolution: The images have varying resolutions but are generally of good quality for processing.
- Formats: The images are primarily in JPEG/PNG format.

Image Variability

The dataset includes a wide range of images with different conditions, such as:
Fire in indoor and outdoor environments.
Fire with different colors, intensities, and smoke levels.
Day and night fire scenes.
Fire in forests, buildings, vehicles, and other environments.
False positives such as bright lights, sunsets, and artificial flames to challenge the model's robustness.

Application in Fire Detection Models

This dataset is suitable for:

✔ CNN-based classification models (Fire vs. Non-Fire).

✔ Object detection models (like YOLO, Faster R-CNN) for locating fire in images.

✔ Segmentation models to highlight fire-affected areas.

✔ Real-time fire detection applications using OpenCV and deep learning.

Potential Preprocessing Steps

Before training a model, some preprocessing techniques can be applied:

- Resizing: Standardizing image sizes for consistent input dimensions.
- Normalization: Scaling pixel values for better model convergence.
- Augmentation: Flipping, rotating, and adding noise to increase dataset diversity.
- Feature Extraction: Using edge detection (Sobel, Canny) to highlight fire characteristics.

Steps in Predicting Fire using CNN

Data Collection and Preprocessing:

- Gather a dataset of images or video footfire containing examples of both fire and non-fire scenes.
- Preprocess the data by resizing images, normalizing pixel values, and augmenting the dataset if necessary to improve model generalization.

Chapter 5

System Analysis

5.1 Problem Definition

Fire incidents can cause severe damage to life, property, and the environment if not detected and addressed promptly. Traditional fire detection methods, such as manual surveillance or basic smoke detectors, often fail to provide early warning in large, complex, or high-risk areas. Delays in detecting a fire can result in catastrophic consequences, including rapid fire spread, loss of valuable assets, and increased risk to human safety. Additionally, false alarms caused by bright lights or other environmental factors can reduce trust in conventional detection systems and slow emergency response.

The AI-Based Fire Detection System using CNN with Mail Alert Notification addresses these challenges by providing a reliable, automated, and intelligent solution for real-time fire monitoring. By combining deep learning with video surveillance, the system can accurately distinguish real fires from false positives and immediately alert nearby individuals through alarms. Furthermore, automated email notifications with video evidence and predictive insights ensure that relevant authorities are informed quickly, even when no one is physically present. This proactive approach minimizes damage, improves response times, and enhances overall fire safety in residential, commercial, and industrial settings.

5.2 Project Description

The AI-Based Fire Detection System using CNN with Mail Alert Notification is an intelligent solution designed to monitor fire hazards in real time. The system captures live video streams from strategically placed cameras, which are processed using OpenCV to extract important visual features. A Convolutional Neural Network (CNN) then analyzes these frames to detect fire accurately, distinguishing it from other bright objects or reflections. Upon detection, the system immediately triggers audible alarms, alerting people nearby and enabling swift evacuation and emergency response.

In addition to on-site alerts, the system automatically sends email notifications to designated recipients through an SMTP module, providing crucial information such as the detection timestamp, a 10-second video clip, and predictive insights about the fire. Detected events are securely stored for future review and analysis, assisting in post-event investigations and safety planning. By integrating AI-based detection, real-time alerts, and automated reporting, this project ensures a proactive, reliable, and efficient approach to fire safety in residential, commercial, and industrial environments.

Module Description

Camera Input Module

The Camera Input Module is responsible for capturing real-time video footage from strategically installed cameras in the monitored environment. These cameras are positioned to cover critical areas, ensuring that every section of the surveillance zone is monitored continuously. The video streams are transmitted to the system, where OpenCV handles real-time frame extraction and preliminary processing. The module supports high-resolution cameras, including night vision capabilities, enabling effective monitoring even in low-light conditions.

To ensure smooth and reliable video acquisition, the module dynamically adjusts frame rates based on network conditions and system load. It also supports multiple camera integration, making it scalable for large areas like industrial zones, forests, or commercial buildings. The captured frames are then forwarded to the Preprocessing Module to enhance image quality and optimize the input for fire detection analysis. This setup ensures minimal delay between capturing footage and detecting potential fire incidents.

Fire Detection Module with CNN

The Fire Detection Module is the core of the system, utilizing a Convolutional Neural Network (CNN) to analyze processed video frames for fire patterns. The CNN model is trained on a comprehensive dataset containing images of fire and non-fire scenarios to ensure accurate detection. It identifies key features such as flame shape, color variations, and movement patterns to differentiate real fire incidents from false positives caused by bright lights, reflections, or other artificial sources.

Once the CNN classifies a frame as containing fire, it immediately triggers the corresponding safety mechanisms, including alarms and email alerts. The detection process runs in real time, minimizing the time between fire occurrence and response activation. Additionally, the system can update the CNN model with new fire data, allowing continuous learning to improve detection accuracy and robustness over time.

Preprocessing Module

The Preprocessing Module enhances the quality of captured video frames before they are analyzed by the CNN. It applies image processing techniques such as resizing, contrast enhancement, noise reduction, and color normalization to standardize the input frames. These steps improve the model's ability to extract relevant features and reduce errors caused by environmental variations like lighting changes.

Advanced techniques such as Gaussian blurring and edge detection are employed to highlight fire-specific characteristics while suppressing background noise. This ensures that the CNN model focuses on critical visual features, such as flame movement and color transitions, improving classification accuracy. By optimizing the input frames, the Preprocessing Module strengthens the overall reliability of the fire detection system.

Feature Extraction Module

The Feature Extraction Module isolates essential visual features from preprocessed frames, making it easier for the CNN to classify fire accurately. Fire exhibits unique characteristics such as irregular shapes, rapid motion, high-intensity brightness, and color variations ranging from yellow to orange and red. This module identifies and extracts these features to enhance detection precision.

Techniques like histogram analysis, color thresholding, and texture analysis are used to differentiate fire from non-fire objects, such as vehicle headlights or artificial lights. The extracted features are then passed to the Fire Detection Module, enabling the CNN to classify frames with higher accuracy. This module ensures that the system maintains low false positives while reliably detecting actual fire occurrences.

Alarm Sound Activation Module

The Alarm Sound Activation Module is responsible for alerting people nearby as soon as fire is detected. It triggers high-decibel audible alarms through external speakers, ensuring that individuals in the affected area are immediately aware of the emergency. This instant response is crucial for safe evacuation and minimizing the risk of injuries or loss of life.

The alarm system can be customized based on the severity of the fire, including dynamic adjustment of sound intensity according to the location and spread of the fire. Additionally, it can be integrated with IoT devices like automated fire sprinklers, which activate alongside the alarm to help control the fire until emergency responders arrive.

SMTP Integration Module

The SMTP Integration Module automatically sends email alerts to designated recipients when fire is detected. Using the Simple Mail Transfer Protocol (SMTP), the system delivers critical notifications to authorities, building security teams, or property owners, even when no one is physically present at the site. Emails include essential details such as detection timestamp, a 10-second video clip, and location information.

The module supports multi-recipient functionality, ensuring that all relevant stakeholders are informed simultaneously. It also maintains a log of all sent alerts for future reference or audits, which is particularly useful in large commercial or industrial settings where rapid, coordinated responses are required. This module ensures timely communication and enhances overall situational awareness during emergencies.

Video Storage and Retrieval Module

The Video Storage and Retrieval Module securely records and stores video clips whenever fire is detected. Each event is saved as a 10-second video, along with metadata such as timestamp, location, and severity. This provides a reliable record for reviewing incidents and conducting post-event investigations to improve preventive measures.

The module also includes a user-friendly interface for retrieving stored footage. It offers search and filtering functionalities based on date, time, location, or severity, making it easy to access specific video clips. These records can also be used for training AI models and refining fire safety strategies, ensuring continuous improvement of the detection system.

5.3 User Interface

The User Interface (UI) of the AI-Based Fire Detection System is designed to be intuitive, interactive, and user-friendly, allowing users to monitor real-time video streams and manage system settings with ease. The interface provides a live display of camera feeds, showing both processed and raw video frames, along with indicators highlighting detected fire regions. Users can access system status, alarm notifications, and detection logs through a clean dashboard, making it simple to track incidents and verify fire alerts.

Additionally, the UI includes features for managing email notifications, viewing stored video clips, and generating reports of past events. Search and filtering options enable users to quickly retrieve specific video recordings based on date, time, or location. The interface is designed for both desktop and mobile use, ensuring accessibility for security personnel and authorities at all times. Overall, the UI enhances situational awareness, facilitates rapid decision-making, and ensures effective interaction with the fire detection system.

Dashboard

The Dashboard serves as the main control panel of the fire detection system, providing users with a comprehensive overview of real-time activities. It displays live video feeds from all connected cameras, highlighting any detected fire regions with visual markers. Users can quickly assess the status of each monitored area, view alarm activations, and track system performance through intuitive graphs and status indicators.

The dashboard also integrates alerts and notifications, ensuring that users can immediately respond to fire incidents. Historical data and recent detection events are summarized for easy analysis, allowing users to monitor trends and evaluate system efficiency. This centralized interface simplifies monitoring large areas and ensures quick situational awareness.

Live Video Monitoring

The Live Video Monitoring section allows users to observe real-time footage from strategically placed cameras. Each feed is processed to highlight fire-prone areas, and visual cues are provided when the system detects potential fire incidents. Users can zoom in, switch between camera feeds, and adjust display settings to focus on critical regions.

Additionally, the system supports multiple camera integration, enabling seamless monitoring of large or multi-level areas such as industrial zones or commercial buildings. The live monitoring interface ensures that users are continuously aware of potential hazards and can make rapid decisions in case of emergencies.

Alarm and Notifications Panel

The Alarm and Notifications Panel displays active alerts triggered by the system. When a fire is detected, this module provides visual and audio indicators, showing the location, severity, and timestamp of the event. Users can acknowledge alerts, view detailed information, and track the status of ongoing incidents directly from this panel.

This panel also integrates with the SMTP module, showing the status of automated email notifications sent to stakeholders. Users can see which recipients have been notified and review email logs for auditing purposes. This ensures that all critical alerts are communicated effectively and timely.

Video Storage and Retrieval Interface

The Video Storage and Retrieval Interface allows users to access previously recorded clips of fire incidents. Each video is tagged with metadata such as timestamp, camera location, and severity of detection, making it easy to filter and search for specific events. Users can play back clips, download recordings, or export them for reporting and investigation purposes.

Additionally, this interface provides an organized library of stored videos, supporting bulk management and review. Users can also use this module for training and improving the CNN model by analyzing past events, ensuring continuous improvement of the fire detection system.

Settings and Configuration Panel

The Settings and Configuration Panel allows users to manage system parameters, such as camera configurations, detection sensitivity, alarm thresholds, and email notification settings. This module ensures that the fire detection system can be tailored to specific environments, optimizing accuracy and responsiveness.

Users can also schedule maintenance checks, update the CNN model, and adjust preprocessing parameters through this interface. The configuration panel provides a secure and organized way to maintain the system, ensuring reliable operation and easy adaptability to changing conditions.

Chapter 6

System Implementation & Testing

System Implementation

The AI-Based Fire Detection System is implemented using Python, leveraging OpenCV for real-time video capture and preprocessing, and TensorFlow for training and running the Convolutional Neural Network (CNN) model. The system first captures video streams from strategically placed cameras, and each frame undergoes preprocessing steps such as resizing, noise reduction, and color normalization to optimize feature extraction. The CNN model then analyzes these frames to detect fire patterns, distinguishing real fire incidents from false positives caused by bright lights or reflections. Once fire is detected, the system triggers immediate alarms and sends automated email notifications with relevant details, including a 10-second video clip, timestamp, and affected location.

The implementation also includes modules for feature extraction, video storage, and retrieval, enabling continuous monitoring and record-keeping for post-event analysis. The SMTP module ensures timely communication to multiple stakeholders, while the user interface provides a real-time dashboard, live video monitoring, and configuration settings for effective system management. By integrating AI-based detection with real-time alerts, automated reporting, and secure video logging, the system offers a robust and scalable solution for fire safety in residential, commercial, and industrial environments.

System Testing

The AI-Based Fire Detection System undergoes rigorous testing to ensure accuracy, reliability, and real-time performance. Functional testing is performed on each module, including camera input, preprocessing, CNN-based fire detection, alarm activation, and SMTP notifications, to verify that they operate as expected under different conditions. Test scenarios include controlled fire simulations, variations in lighting, and the presence of bright non-fire objects to evaluate the system's ability to correctly identify fire incidents and avoid false positives.

Performance testing is also conducted to assess system responsiveness, including the time taken to detect fire and trigger alerts. The system's video storage and retrieval module is tested for proper recording and playback, ensuring that 10-second clips are accurately stored and easily accessible. Email notifications are verified for correct delivery and content. Overall, testing ensures that the system performs reliably in real-world scenarios, providing timely alerts and effective fire monitoring across residential, commercial, and industrial settings.
There are different types of system testing

- Unit Testing
- Integration Testing
- Validation Testing
- Acceptance Testing

Unit Testing

Unit Testing focuses on verifying the functionality of individual components or modules of the fire detection system. Each module, such as the Camera Input Module, Preprocessing Module, CNN Fire Detection Module, Alarm Activation Module, and SMTP Notification Module, is tested independently to ensure that it performs its intended function accurately. For instance, the Camera Input Module is tested for proper frame capture, while the CNN module is validated for accurate fire classification.

This testing helps identify and fix issues at the earliest stage of development, reducing the risk of errors propagating to other modules. Unit testing also ensures that individual modules meet performance requirements, such as processing speed, detection accuracy, and email delivery reliability, before they are integrated into the complete system.

Integration Testing

Integration Testing is performed to verify that different modules of the system work seamlessly together. After unit testing, the Camera Input Module, Preprocessing Module, Fire Detection Module, Alarm Module, SMTP Module, and Video Storage Module are combined, and their interactions are tested to ensure smooth data flow and correct communication between modules. For example, a video frame captured by the camera should be preprocessed, analyzed by the CNN, and, if fire is detected, trigger both the alarm and email notification correctly.

This testing identifies issues arising from module interactions, such as delays, data loss, or incorrect alarm triggers, which may not be evident during unit testing. It ensures that the integrated system functions reliably as a whole, maintaining accuracy, speed, and consistency in real-time fire detection and alerting.

Validation Testing

Validation Testing ensures that the system meets the specified requirements and functions according to user expectations. The fire detection system is tested under various real-world scenarios, including different lighting conditions, indoor and outdoor environments, and the presence of artificial lights or reflective surfaces. The goal is to confirm that the system accurately detects fires, minimizes false alarms, and sends timely alerts to designated recipients.

This testing also evaluates system performance in terms of response time, video recording quality, and notification delivery. By validating against functional and non-functional requirements, the process confirms that the system is reliable, accurate, and suitable for deployment in residential, commercial, and industrial settings.

Acceptance Testing

Acceptance Testing is the final stage of testing, where the system is evaluated by end-users or stakeholders to ensure it fulfills their operational needs. Users assess the system's overall functionality, including real-time monitoring, alarm activation, automated email notifications, and video storage and retrieval, to confirm that it meets their expectations for fire safety and emergency response.

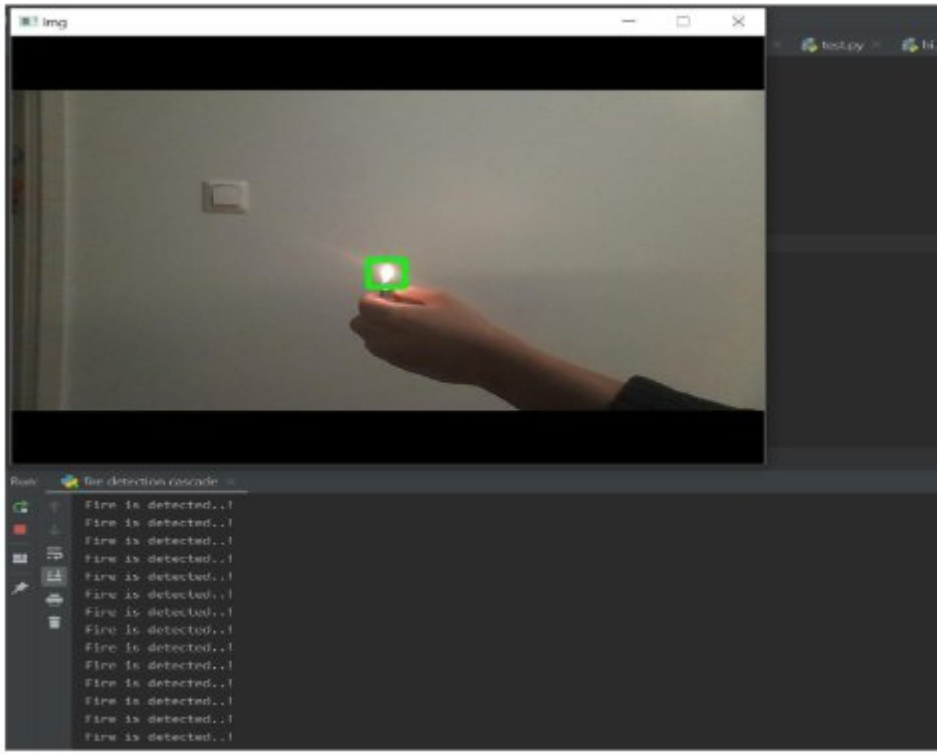
During this phase, feedback from users is collected and any necessary adjustments are made to optimize usability, interface design, and alert effectiveness. Successful acceptance testing confirms that the AI-Based Fire Detection System is ready for deployment and capable of providing reliable, real-time fire monitoring and alerts in practical environments.

Chapter 7

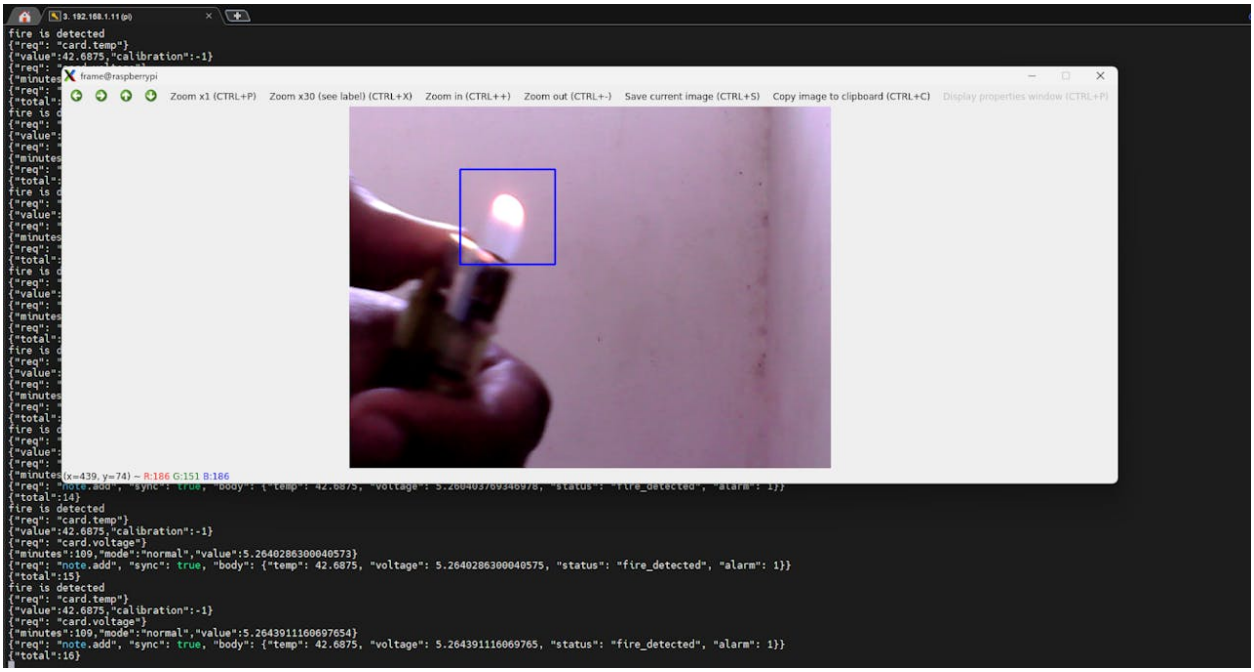
Screen Layout & Reports

SCREEN LAYOUT

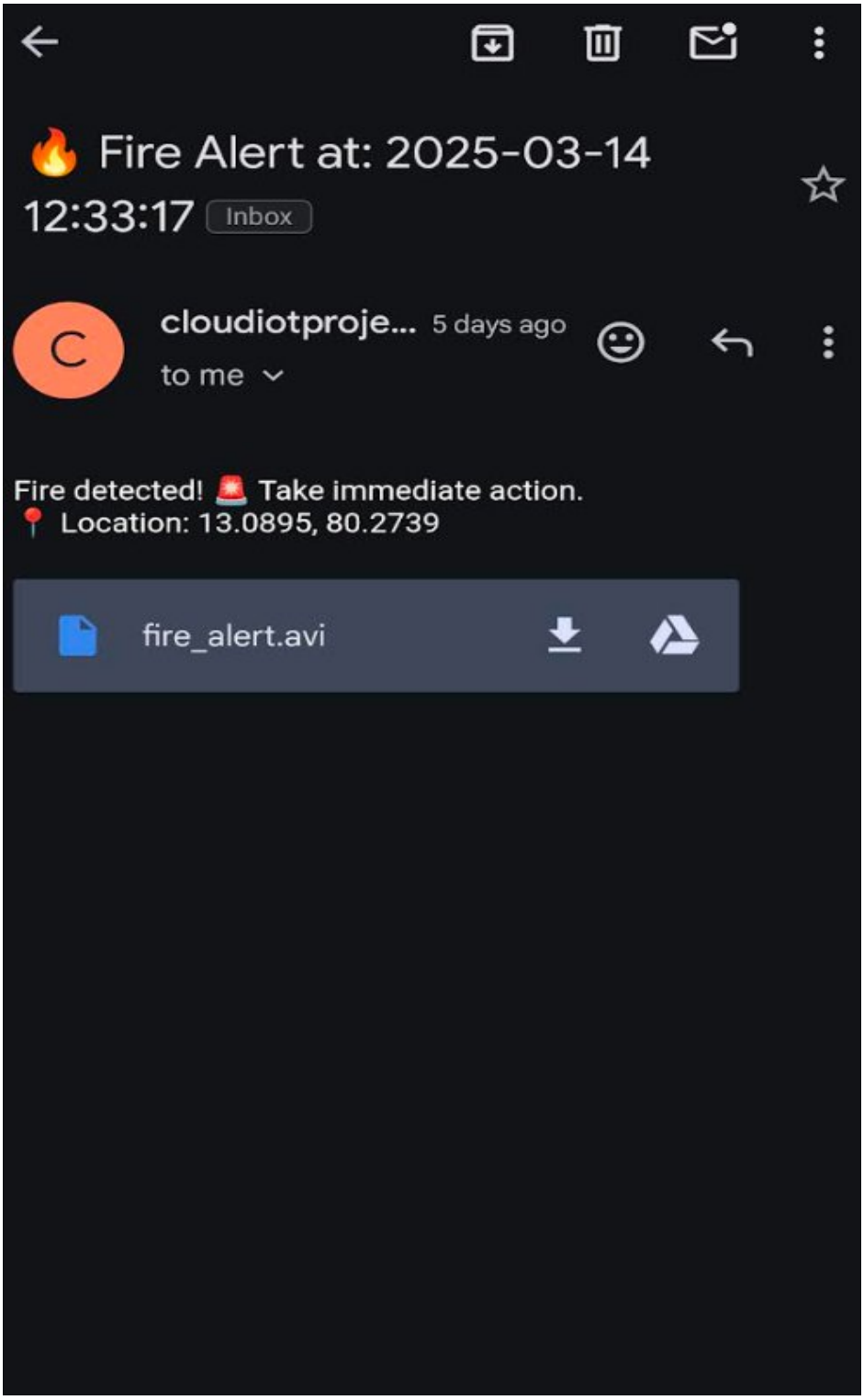
Fire Detection



Classify Fire and Predict



Mail Alert


Fire Alert at: 2025-03-14 12:33:17
Inbox
cloudiotproje... 5 days ago
to me
Fire detected! Take immediate action.
Location: 13.0895, 80.2739
fire_alert.avi

Chapter 8

Limitation / Suggested Improvements

While the AI-Based Fire Detection System using CNN provides accurate and real-time fire monitoring, it has some limitations. The system's performance can be affected by extreme lighting conditions, dense smoke, or heavy rainfall, which may reduce detection accuracy. Additionally, the reliance on camera coverage means that areas outside the camera's field of view remain unmonitored, and false positives may still occur due to highly reflective surfaces or sudden bright light changes. The system also requires continuous power supply and stable network connectivity for real-time video transmission and email notifications, which may pose challenges in remote or low-resource locations.

To enhance the system's effectiveness, several improvements can be considered. Integrating thermal or infrared cameras can improve detection accuracy in low-visibility conditions such as smoke, fog, or nighttime environments. Incorporating IoT sensors for temperature, smoke, or gas detection can provide multi-modal fire monitoring and reduce false positives. Additionally, optimizing the CNN model with larger and more diverse datasets, and implementing edge computing for faster local processing, can further enhance detection speed and reliability. Expanding the system's scalability for multiple sites and providing mobile app integration for instant alerts can also improve usability and overall fire safety management.

Chapter 9

Conclusion

The AI-Based Fire Detection System using CNN with Mail Alert Notification provides an intelligent and efficient solution for real-time fire monitoring. By combining deep learning with OpenCV-based video processing, the system can accurately detect fire patterns, distinguish them from false positives, and immediately alert nearby individuals through audible alarms. The integration of automated email notifications ensures that relevant authorities are informed promptly, even when personnel are not present, enhancing overall safety and emergency response.

The system also offers video storage and retrieval for post-event analysis, helping improve preventive measures and refine fire safety strategies. With its modular design, scalability, and potential for future enhancements, the project demonstrates a proactive approach to fire hazard management. It is suitable for residential, commercial, and industrial environments, significantly reducing risks and providing reliable, real-time fire detection and alert capabilities.

.

Bibliography

Referred Books

- "Automate The Boring Stuff With Python, 2Nd Edition: Practical Programming For Total Beginner", SL Swegart, 2020.
- "Learning Python 5ed: Powerful Object-Oriented Programming" O'Relly – 2014.
- "Elements of Programming Interviews in Python: The Insiders' Guide", Adan Aziz, Tsung-Hsen Lee – 2021.

Referred websites

- https://www.tensorflow.org/about
- https://en.wikipedia.org/wiki/TensorFlow
- https://www.infoworld.com/article/3278008/what-is-tensorflow-the-machine-learning-library-explained.html
- https://www.python.org/about/
- https://www.w3schools.com/python/python_intro.asp
- https://www.geeksforgeeks.org/python-programming-language/